5 Structured Query Language – SQL

Was ist SQL?

- Datenbanksprache
- Deskriptiv (das gewünschte Ergebnis beschreibend)
- Basis: Relationenalgebra bzw. Mengenlehre
- Aufgaben
  - Definition relationaler Datenbanken
  - Bearbeitung und Auswertung der Daten

Merkmale von SQL

- Plattformunabhängigkeit
- Normung
  - American National Standards Institute (ANSI)
  - International Standards Organisation (ISO)
- Große Verbreitung
- Kleiner, aber mächtiger Sprachumfang

5 SQL – Teilbereiche

- Data Definition Language (DDL)
  - Definition der Datenbankstruktur
- View Definition Language (VDL)
  - Definition von Sichten auf die Daten
- Data Manipulation Language (DML)
  - Manipulation und Retrieval von Datensätzen
- Data Control Language (DCL)
  - Definition von Zugriffsrechten
- Data Storage Definition Language (DSDL)
  - Definition der physischen Speicherstruktur

5 SQL Standard

- Trotz Standardisierung Unterschiede in den verschiedenen Implementierungen
- DDL, VDL und DML weitgehend konform
- Unterschiede in DCL und DSDL

  ➔ Handbücher und Dokumentationen
- Es gibt Erweiterungen zum SQL
  z.B. im Datawarehouse-Kontext (Cube/Rollup-Operator)

5 Erstellung einer Datenbank

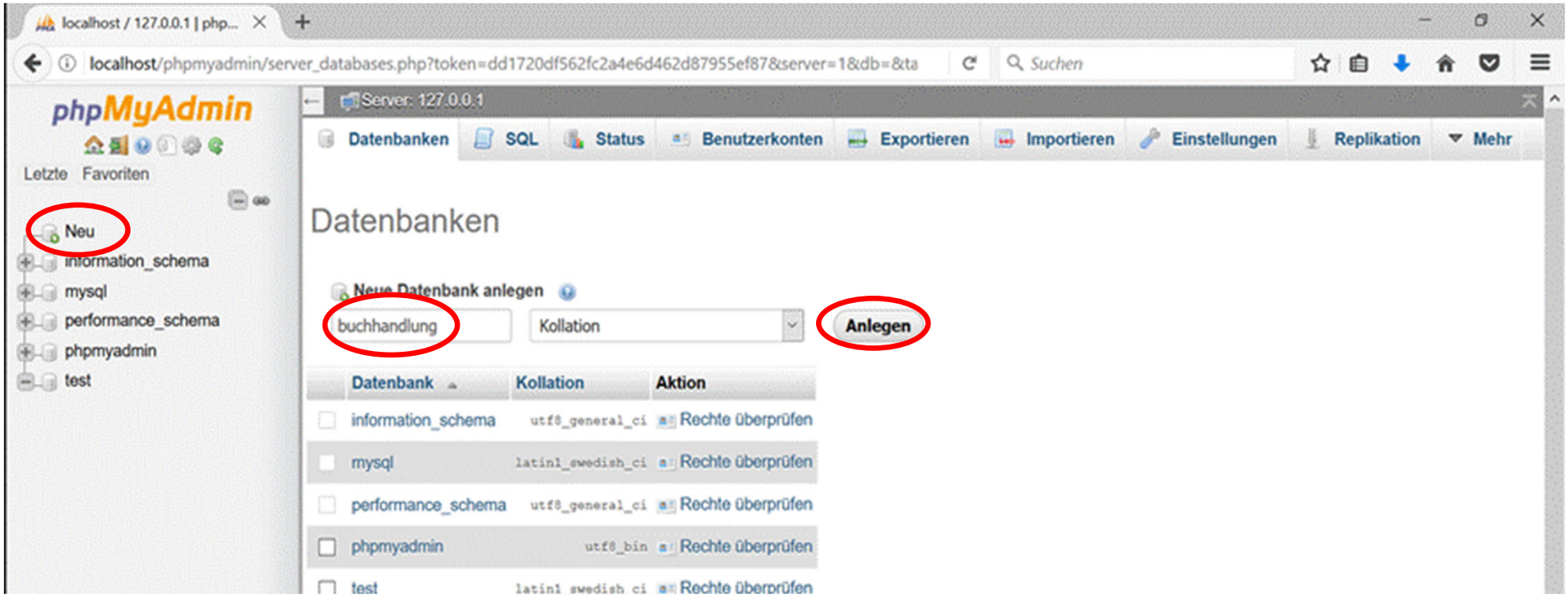
- Datenbankmanagementsysteme können mehrere Datenbanken parallel verwalten

```
CREATE DATABASE <dbname>;
```

```
USE <dbname>;
```

5 Erstellung einer Datenbank – phpMyAdmin

- Menü links: Neu → Namen eingeben → Anlegen
- SQL Befehl: `CREATE DATABASE buchhandlung;`



5 Data Description Language (DDL)

- Definition der Datenbankstruktur
  - Erzeugen
  - Ändern
  - Löschen
- von Datenbanktabellen

5 Erzeugen von Tabellen

```
CREATE TABLE <dbname>.<tabellenname>  (
  <spaltenname> DATENTYP [NOT NULL] [{AUTO_INCREMENT | DEFAULT WERT}],
                ...
  <spaltenname> DATENTYP [NOT NULL] [{AUTO_INCREMENT | DEFAULT WERT}],
 [[CONSTRAINT <constraintname>] PRIMARY KEY (<spaltenname> [,… <sp.>]),]
 [[CONSTRAINT <constraintname>] FOREIGN KEY (<spaltenname> [, … <sp.>])
      REFERENCES <tabellenname> [(<spaltenname>, … <spaltenname>)]
         [ON   {DELETE | UPDATE}
           {RESTRICT | CASCADE | SET NULL | SET DEFAULT | NO ACTION]},]
 [[CONSTRAINT <constraintname>] CHECK <bedingung>,]
 [[CONSTRAINT <constraintname>] UNIQUE (<spaltenname>, … <sp.>)]
 [COMMENT = '']
);
```

5 Datentypen

Datentyp	Inhalt	Beispiele
INTEGER	ganze Zahl	0 / -5 / 1500
REAL / DOUBLE	Fließkommazahlen	0.0 / 34.23 / -13.4534
NUMERIC(x,y)	Festkommazahlen	0.0 / 34.23 / -13.4534
TEXT	Text unbegrenzter Länge	„Lorem ipsum dolor…“
VARCHAR(max)	Zeichenkette variabler Länge	‘aA?‘ / ‘Test‘ / ‘‘
CHAR(fix)	Zeichenkette fixer Länge	‘aA?‘ / ‘Test‘ / ‘‘
DATE	Datumswert	2021-12-08
TIME	Tageszeit (ohne Datum)	08:47:51
TIMESTAMP	Datum mit Uhrzeit	2038-01-19 03:14:07
… WITH TIME ZONE	Zeitdaten mit Zeitzonendaten	08:47:51+01
BOOLEAN	Logischer Wert	TRUE / FALSE

https://www.postgresql.org/docs/current/datatype.html

5 Erzeugen von Tabellen durch Abfragen

```
CREATE TABLE <tabellenname> AS <select befehl>;
```

- „Kopieren“ der Struktur
- Automatisches Füllen mit Datensätzen

5 Ändern von Tabellen

- Nur Änderungen ohne Informationsverlust

ALTER TABLE <tabellenname> **ADD COLUMN** <spaltendefinition>;

ALTER TABLE <tabellenname> **DROP** <spaltenname>;

ALTER TABLE <tabellenname> **RENAME** <spaltenname> **TO** <neuer_name>;

ALTER TABLE <tabellenname> **ALTER** <spaltenname> TYPE DATENTYP;

ALTER TABLE <tabellenname> **ALTER** <spaltenname> **SET DEFAULT** <wert>;

ALTER TABLE <tabellenname> **ALTER** <spaltenname> **DROP DEFAULT**;

ALTER TABLE <tabellenname> **ALTER** <spaltenname> **SET NOT NULL**;

ALTER TABLE <tabellenname> **ALTER** <spaltenname> **DROP NOT NULL**;

https://www.postgresql.org/docs/current/sql-altertable.html

5 Ändern von Tabellen

- Nur Änderungen ohne Informationsverlust

```
ALTER TABLE <tabellenname> ADD CONSTRAINT <constraintname> <definition>;
ALTER TABLE <tabellenname> DROP CONSTRAINT <constraintname>;
```

Mögliche Definitionen eines Constraints sind:

```
… PRIMARY KEY (<spaltenname(n)>)

… FOREIGN KEY (<spaltenname>)
  REFERENCES (<spaltenname(n)>)
  ON DELETE <Keyword zu RI>
  ON UPDATE <Keyword zu RI>;
```

https://www.postgresql.org/docs/current/sql-altertable.html

5 Referentielle Integrität

- RESTRICT
  - ⇒ Kein Löschen / Ändern
- CASCADE
  - ⇒ Kaskadiertes Löschen / Ändern
- SET NULL
  - ⇒ Referenzen werden auf NULL gesetzt
- SET DEFAULT
  - ⇒ Referenzen werden auf DEFAULT-Wert gesetzt
- NO ACTION
  - ⇒ Keine Aktion (wie RESTRICT)

5 Referentielle Integrität Beispiel



- ON DELETE CASCADE

kunden

id	name	postleitzahl_FK
1	Schmitt	10000
2	Müller	10000
3	Maier	79312

orte

postleitzahl	name
10000	Musterhausen
79312	Freiburg

löschen

kunden

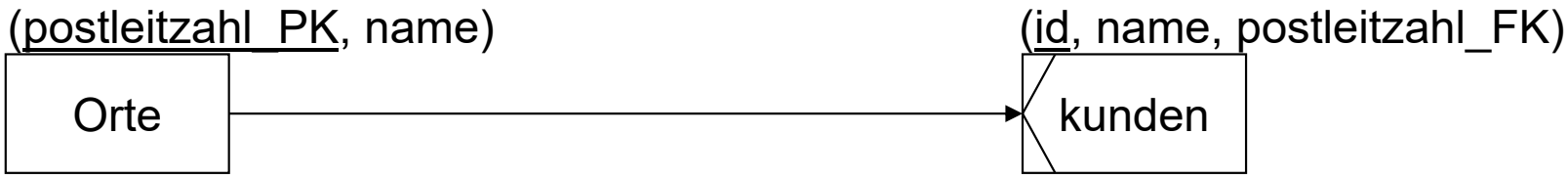
id	name	postleitzahl_FK
3	Maier	79312

automatische Löschung in Kindtabelle

orte

postleitzahl	name
79312	Freiburg

5 Referentielle Integrität Beispiel



- ON UPDATE CASCADE

kunden

id	name	postleitzahl_FK
1	Schmitt	10000
2	Müller	10000
3	Maier	79312

orte

postleitzahl	name
10000	Musterhausen
79312	Freiburg

Änderung

kunden

id	name	postleitzahl_FK
1	Schmitt	?
2	Müller	
3	Maier	9312

automatische Änderung in Kindtabelle

orte

postle zahl	name
99999	Musterhausen
79312	Freiburg

5 Referentielle Integrität Beispiel



- ON DELETE SET NULL

kunden

id	name	postleitzahl_FK
1	Schmitt	10000
2	Müller	10000
3	Maier	79312

orte

postleitzahl	name
10000	Musterhausen
79312	Freiburg

löschen

kunden

id	name	postleitzahl_FK
1	Schmitt	NULL
2	Müller	NULL
3	Maier	9312

automatisch NULL in Kindtabelle

orte

postleitzahl	name
79312	Freiburg

5 Referentielle Integrität Beispiel



- ON UPDATE SET NULL

kunden

id	name	postleitzahl_FK
1	Schmitt	10000
2	Müller	10000
3	Maier	79312

orte

postleitzahl	name
10000	Musterhausen
79312	Freiburg

Änderung

kunden

id	name	postleitzahl_FK
1	Schmitt	NULL
2	Müller	NULL
3	Maier	9312

automatisch NULL in Kindtabelle

orte

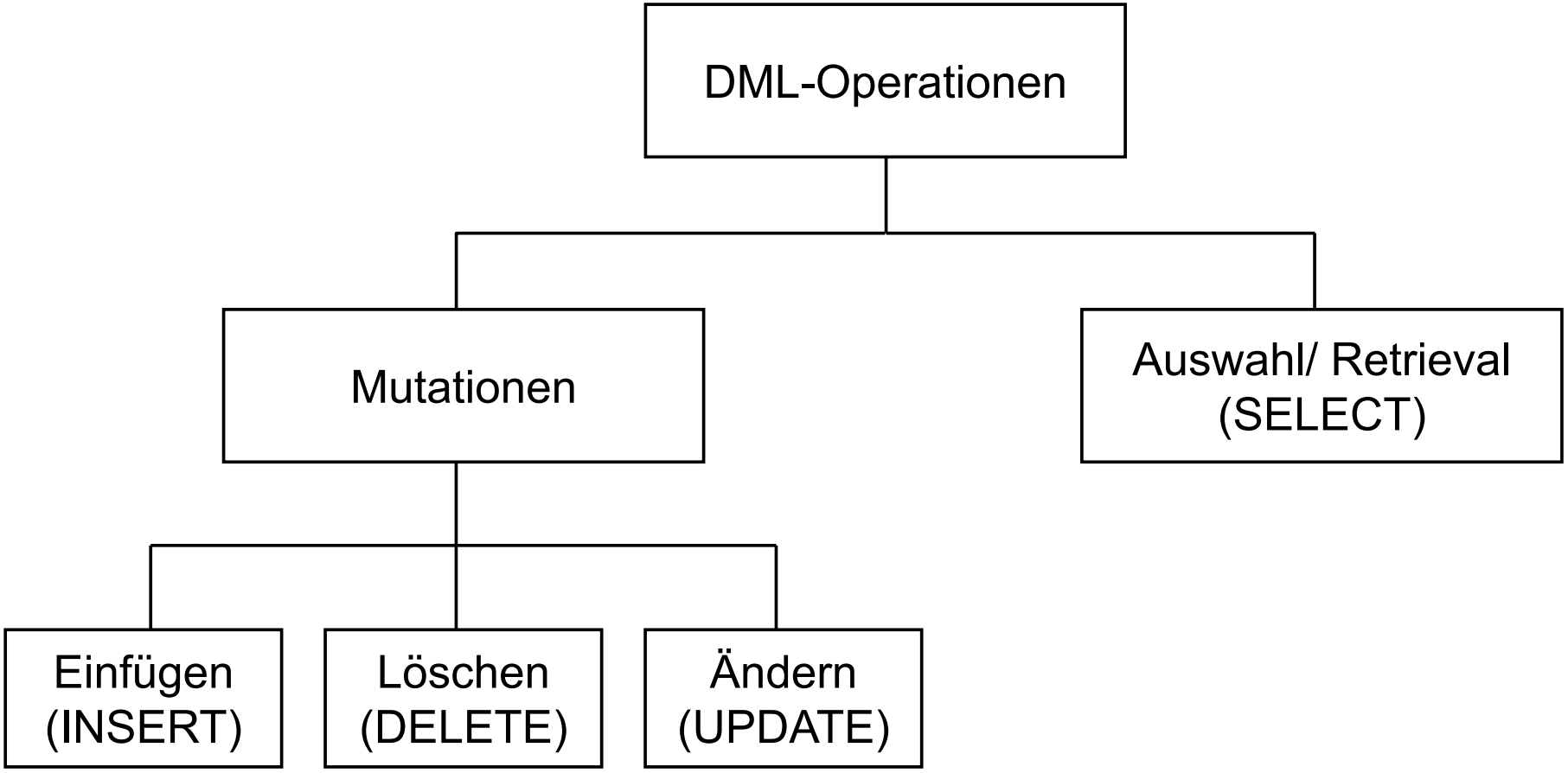
postle zahl	name
99999	Musterhausen
79312	Freiburg

5 Löschen von Tabellen

```
DROP TABLE <tabellenname>;
```

- Berücksichtigung von Abhängigkeiten
- Sicherstellung der referentiellen Integrität

5 Data Manipulation Language



5 Einfügen von Datensätzen

```
INSERT INTO <tabellenname> [(<spaltenname>, ...)]
   VALUES (<werteliste>)[,(<werteliste>),…,(<werteliste>)];
```

- Manche DBMS unterstützen nur ein Datensatz gleichzeitig
- Länge der Werteliste muss der Länge der Liste der Spaltennamen entsprechen

```
INSERT INTO <tabellenname> [(<spaltenname>, ...)]
   AS <select befehl>;
```

5 Löschen von Datensätzen

```
DELETE FROM <tabellenname>
   [WHERE <bedingung>];
```

- Berücksichtigung von Abhängigkeiten
- Sicherstellung der referentiellen Integrität
- Potentiell kaskadiertes Löschen

  ⇒ ON DELETE CASCADE

5 Ändern von Datensätzen

```
UPDATE <tabellenname>
   SET <spaltenname> = <ausdruck> [,<spaltenname> = <ausdruck> ...]
   [WHERE <bedingung>];
```

- Berechnungen in Ausdrücken möglich
- Berücksichtigung von Abhängigkeiten
- Sicherstellung der referentiellen Integrität

Datenretrieval

```
SELECT [DISTINCT] {  <tabellenname>.* |
                     <tabellenname>.<attributname> [AS <alias>]|
                     <ausdruck> [AS <alias>] }
                     [,  ...]
   FROM <tabellenname> [<alias>]
         [LEFT JOIN | RIGHT JOIN | NATURALJOIN | JOIN |
             FULL OUTER JOIN ] [USING | ON] [, ...]
   [WHERE <bedingung>]
   [GROUP BY {<tabellenname>.<attributname> | <alias>} [, ...]]
   [HAVING <bedingung>]
   [UNION <select befehl>]
   [ORDER BY {<tabellenname>.<attributname> | <alias>} [ASC|DESC]
              [, ...]]
;
```

Bem.: FULL OUTER JOIN ist in mysql, mariadb nicht verfügbar.

5 Bestandteile Select-Befehl

- **SELECT (Selection)**
  Auswahl der Attribute/Tabellenspalten
- **DISCTINCT**
  Nur Anzeige unterschiedlicher Datensätze
- ***
  Auswahl aller Attribute einer Tabelle
- **AS**
  Umbenennung eines Attributes / eines Ausdrucks
- **<Ausdruck>**
  Berechnungsvorschrift / Formel
- **FROM … Kreuzprodukt / Join**
  Auswahl der im Select-Befehl verwendeten Tabellen

5 Bestandteile Select-Befehl

- **WHERE (Restriktion)**
  Definition der Bedingungen zur Auswahl der Datensätze
  Ausdruck kann nur die Werte wahr oder falsch annehmen
- **GROUP BY**
  Spalten, nach denen gruppiert werden soll
  alle Spalten, die nicht in Aggregatfunktionen verwendet werden*
- **HAVING**
  Überprüfung der Eigenschaften einer Gruppe
- **UNION**
  Vereinigung der Ergebnisse zweier Select-Befehle
- **ORDER BY**
  Definition der Sortierung der Ergebnisdatensätze

* Manche DBMS fordern dies nicht. ➔ Vorteile & Probleme

5 Operatoren in Where/Having-Bedingungen

- >; >= ; =; =<; <; <>
- AND; OR; NOT
- IS (NOT) NULL: Überprüfung auf den Wert (NOT) NULL
- IN: Mengenoperator
  (Beispiel Menge: SELECT <1 Attribut> FROM …)
- LIKE: Vergleichsoperator für Zeichenketten
  - _: Ein beliebiges Zeichen
  - %: beliebige Zeichenfolge
- BETWEEN … AND : Intervallangabe

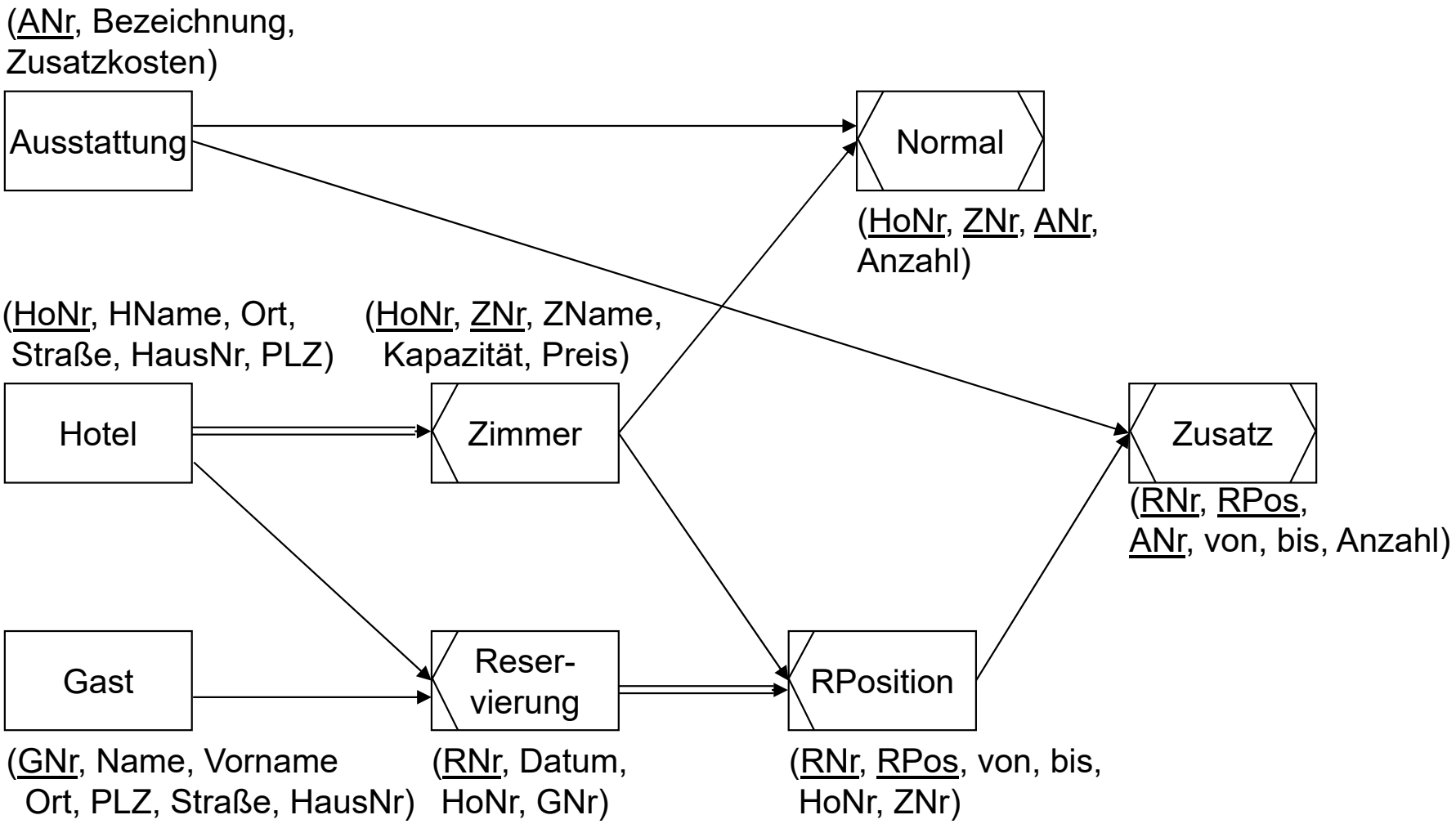
5 Verbund / Join

- Unterabfragen
- UNION
- Spezielle Joins
  - LEFT JOIN
  - RIGHT JOIN
  - NATURAL JOIN
- Join-Bedingung
  - … JOIN … USING (Attribut)
  - … JOIN … ON (Bedingung)

5 Aggregatfunktionen

- COUNT
- SUM
- MIN
- MAX
- AVG

5 Anwendungsbeispiel



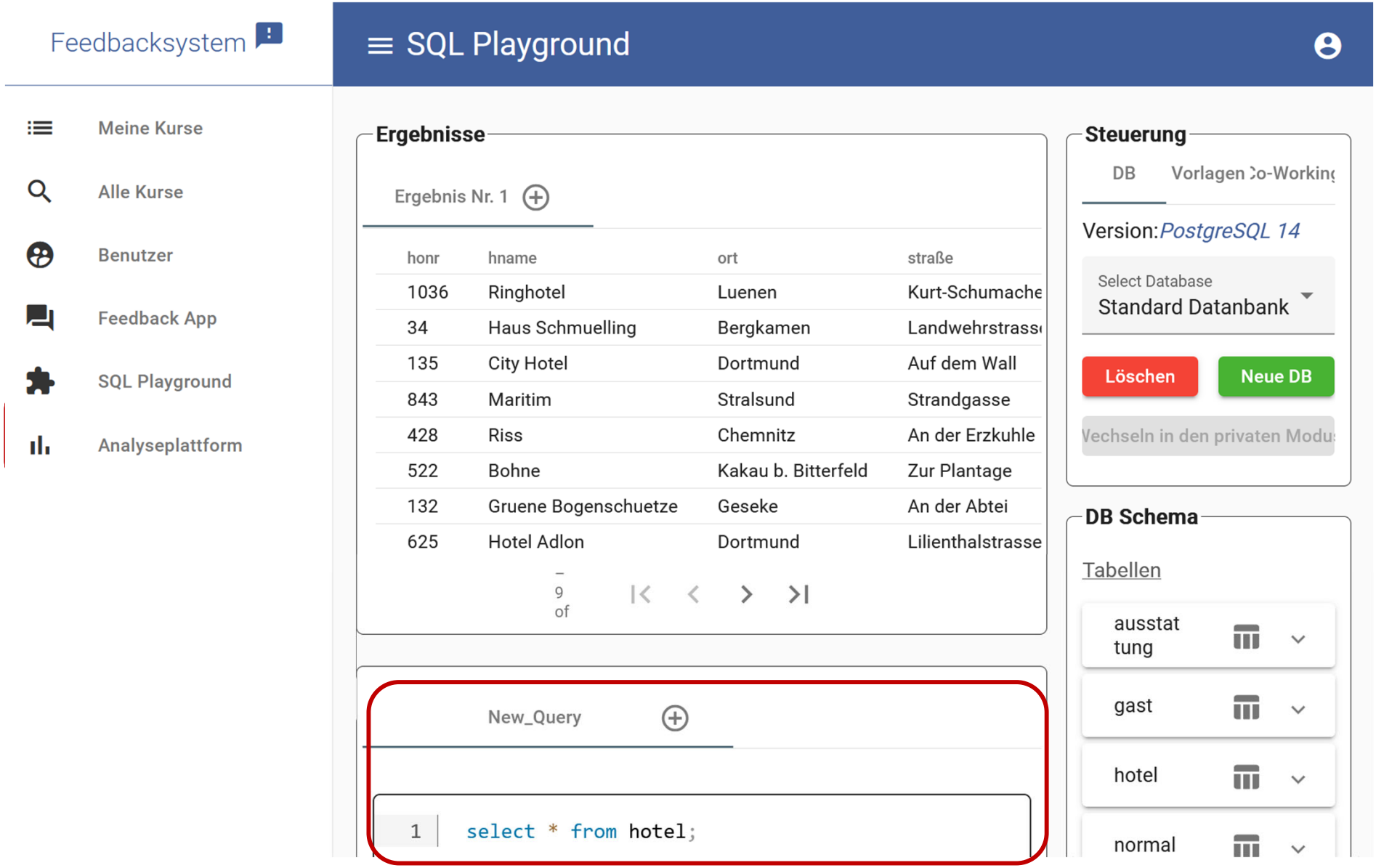
5 SQL-Query im Browser ausführen



5 SQL-Query im Browser ausführen



5 SQL-Query im Browser ausführen



5 SELECT

- Alle Hotels mit allen Attributen

```
SELECT *
FROM hotel;
```

- Alle Hotels mit Hotelnamen und Orten

```
SELECT hname, ort
FROM hotel;
```

- Hotels aus Gießen

```
SELECT hname
FROM hotel
WHERE ort = 'Gießen';
```

5 Verbund über WHERE-Klausel

- Alle Hotels mit ihren Zimmern

```
SELECT hotel.honr, hotel.hname, zimmer.znr
FROM hotel, zimmer
WHERE hotel.honr = zimmer.honr;
```

- Alle Hotels mit ihren Zimmern mit Alias-Verwendung

```
SELECT h.honr, h.hname, z.znr
FROM hotel h, zimmer z
WHERE h.honr = z.honr;
```

- Alle Ausstattungen, die in Hotels verwendet werden

```
SELECT h.honr, h.hname, n.anr, a.bezeichnung
FROM hotel h, normal n, ausstattung a
WHERE h.honr = n.honr and n.anr = a.anr;
```

5 Verbund über JOIN

Joins sind effizienter als ein Verbund mit where.

- Alle Hotels mit ihren Zimmern

```
SELECT h.honr, h.hname, z.znr
FROM hotel h JOIN zimmer z ON h.honr = z.honr;
```

```
SELECT h.honr, h.hname, z.znr
FROM hotel h JOIN zimmer z USING (honr);
```

- Alle Ausstattungen, die in Hotels verwendet werden

```
SELECT h.honr, h.hname, n.anr, a.bezeichnung
FROM hotel h JOIN normal n ON h.honr = n.honr
             JOIN ausstattung a ON n.anr = a.anr;
```

```
SELECT h.honr, h.hname, n.anr, a.bezeichnung
FROM hotel h JOIN normal n USING (honr)
             JOIN ausstattung a USING (anr);
```

5 Unterdrückung gleicher Datensätze

- Alle verschiedenen Ausstattungen, die in Hotels verwendet werden

```
SELECT DISTINCT h.honr, h.hname, n.anr, a.bezeichnung
FROM hotel h JOIN normal n USING (honr)
             JOIN ausstattung a USING (anr);
```

Häufiger Anfängerfehler: Es wird `DISTINCT` an Stellen verwendet, wo `GROUP BY` verwendet werden sollte.

5 Sortierung

- Alle verschiedenen Ausstattungen, die in Hotels verwendet werden, aufsteigend sortiert nach honr

```
SELECT DISTINCT h.honr, h.hname, n.anr, a.bezeichnung
FROM hotel h JOIN normal n USING (honr)
             JOIN ausstattung a USING (anr)
ORDER by h.honr ASC;
```

- Alle Zimmer mit ihrer Normalausstattung sortiert,
  - erst aufsteigend nach Hotelnummer,
  - dann absteigend nach Zimmernummer

```
SELECT z.znr, z.zname, z.honr, n.anr
FROM zimmer z JOIN normal n USING (znr, honr)
ORDER BY z.honr ASC, z.znr DESC;
```

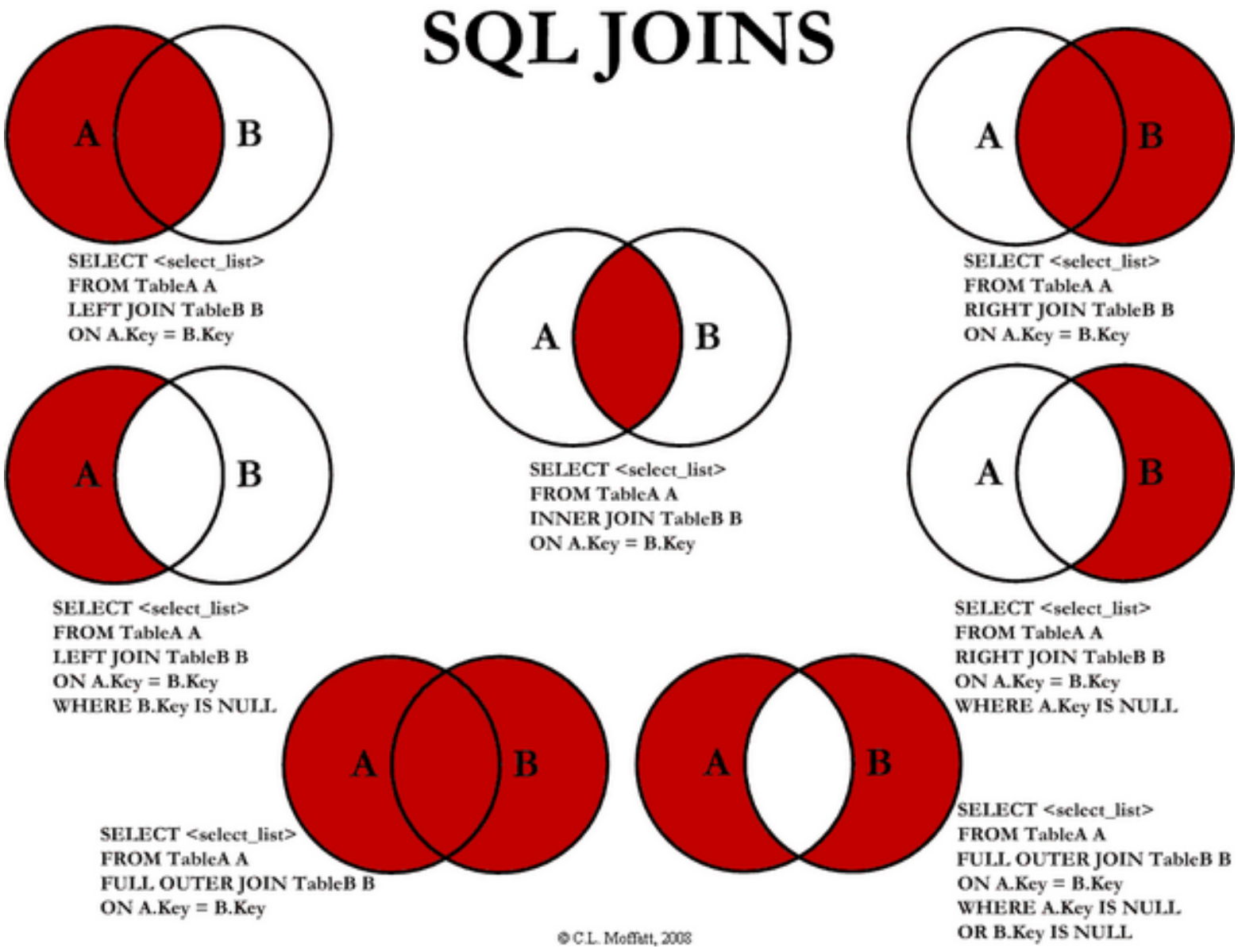
5 Verbund über JOIN

Joins sind effizienter als ein Verbund mit where.

- Auflistung aller Gäste mit ihren Reservierungen, auch wenn diese keine Reservierungen haben

```
SELECT g.gnr, g.name, r.rnr
FROM gast g LEFT JOIN reservierung r USING (gnr);
```

5 Relationenalgebra – Join-Arten



5 Verwendung von Wildcards

- Alle Zimmer mit ihrer Normalausstattung mit allen Attributen

```
SELECT *
FROM zimmer z JOIN normal n USING (znr, honr);
```

- Alle Zimmer mit ihrer Normalausstattung nur mit allen Attributen aus Zimmer

```
SELECT zimmer.*
FROM zimmer z JOIN normal n USING (znr, honr);
```

5 Verwendung von Wildcards

% Beliebige Anzahl an Zeichen
_ Genau ein Zeichen

- Alle Hotels aus Gießen

```
SELECT hname
FROM hotel
WHERE ort = 'Gießen';
```

- Alle Hotels aus Städten mit G am Anfang

```
SELECT hname
FROM hotel
WHERE ort LIKE 'G%';
```

- Alle Hotels mit einem i an zweiter Stelle (Gießen, Dillenburg, …)

```
SELECT hname
FROM hotel
WHERE ort LIKE '_i%';
```

5 Unterabfragen

- Gäste, die für das Hotel Adlon reserviert haben

```
SELECT g.name, g.vorname
FROM gast g JOIN reservierung r USING (gnr)
            JOIN rposition rp USING (rnr)
            JOIN hotel h ON h.honr = rp.honr
WHERE h.hname='Hotel Adlon';
```

- Gäste, die noch nie für das Hotel Adlon reserviert haben

```
SELECT g.name, g.vorname
FROM gast g
WHERE g.gnr NOT IN (
         SELECT g.gnr
         FROM gast g JOIN reservierung r USING (gnr)
            JOIN rposition rp USING (rnr)
            JOIN hotel h ON h.honr = rp.honr
         WHERE h.hname='Hotel Adlon')
;
```

5 SELFJOIN

- Welche Hotels liegen im gleichen Ort?

```
SELECT a.hname, b.hname, a.ort
FROM hotel a, hotel b
WHERE a.ort=b.ort;
```

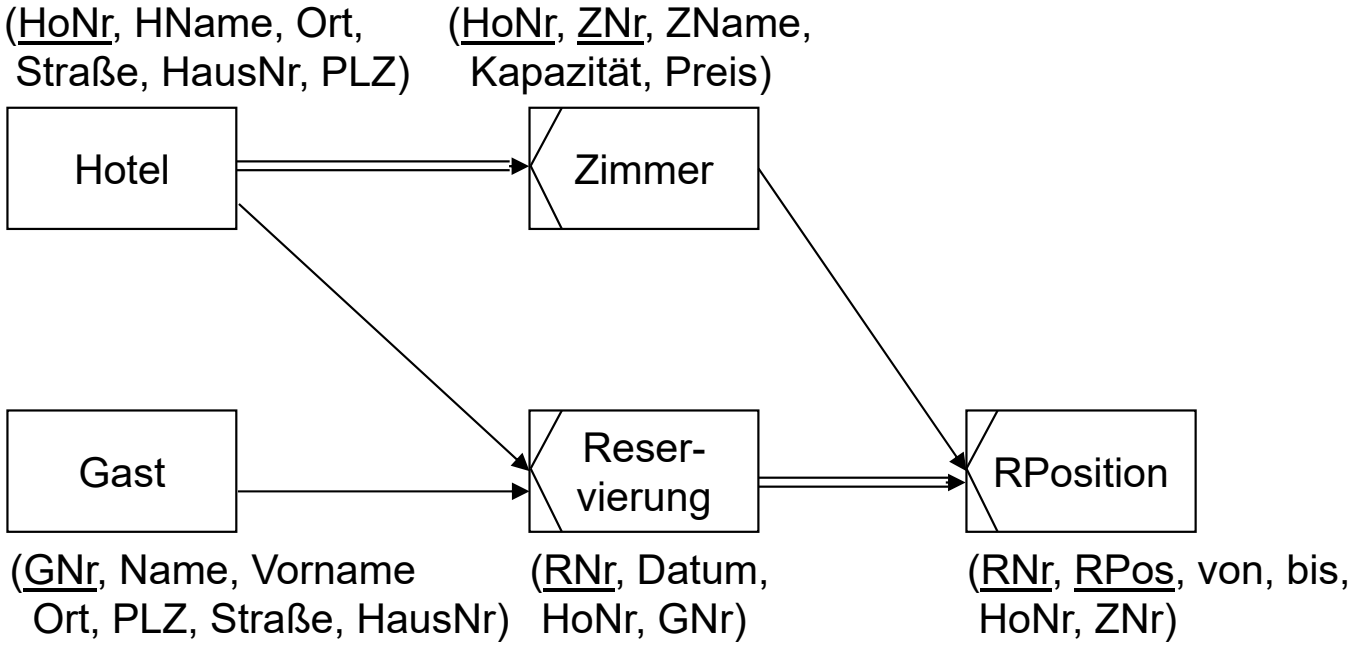
- Welche Hotels liegen im gleichen Ort (ohne gleiche Hotels)?

```
SELECT a.hname, b.hname, a.ort
FROM hotel a, hotel b
WHERE a.ort=b.ort AND a.honr<>b.honr;
```

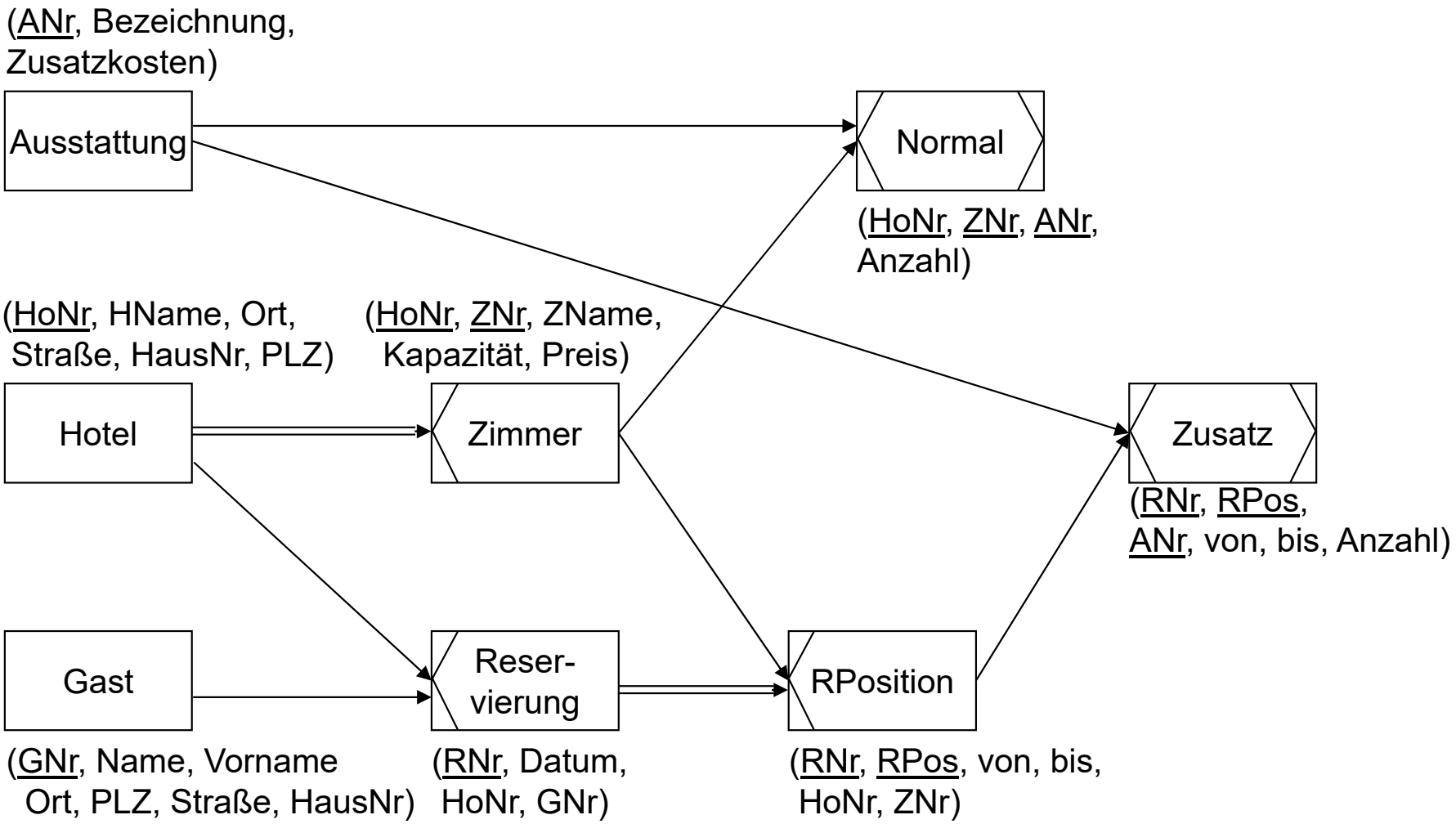
5 Mehrfachverwendung einer Tabelle

- Welcher Gast hat in welchem Hotel (entgegennehmendes Hotel) für welches Hotel reserviert?

```
SELECT g.gnr, g.name, h.honr, h.hname, e.honr, e.hname
FROM gast g JOIN reservierung r USING (gnr)
            JOIN hotel e ON e.honr = r.honr
            JOIN rposition rp USING (rnr)
            JOIN hotel h ON h.honr = rp.honr;
```



5 Anwendungsbeispiel – Zur Erinnerung



5 Berechnungen

- Auflistung der Kosten aller Reservierungspositionen für jeden Kunden

```
SELECT g.gnr, g.name, z.preis*(rp.bis-rp.von)
FROM gast g JOIN reservierung r USING (gnr)
            JOIN rposition rp USING (rnr)
            JOIN zimmer z ON z.honr = rp.honr AND z.znr = rp.znr;
```

- Auflistung der Kosten aller Reservierungspositionen für jeden Kunden mit Spaltenbenennung

```
SELECT g.gnr, g.name, z.preis*(rp.bis-rp.von) AS kosten
FROM gast g JOIN reservierung r USING (gnr)
            JOIN rposition rp USING (rnr)
            JOIN zimmer z ON z.honr = rp.honr AND z.znr = rp.znr;
```

5 SQL – Übungen

Gegeben sind folgende Tabellen:

Person

Personalnr	Name	Gehalt	Ort	Abteilung
100	Maier	3800.00 €	Künzelsau	1
101	Müller	4100.00 €	Künzelsau	2
102	Schmidt	3500.00 €	Öhringen	1
103	Schulze	4600.00 €	Heilbronn	3
104	Hinz	4200.00 €	Heilbronn	3
105	Kunz	4000.00 €	Schwäbisch Hall	5

Abteilung

Abteilungsnr	Abteilungsname
1	EDV
2	Produktion
3	Vertrieb
4	Verwaltung
5	Entwicklung

5 SQL – Übungen

1) Geben Sie für folgende SQL-Abfragen an, welche Tabelle als Ergebnis erzeugt wird (Tabelle mit Überschriften):

a)
```
SELECT DISTINCT Ort
    FROM Person
```

b)
```
SELECT Name
    FROM Person
    WHERE Abteilung = 1
    ORDER BY Name DESC
```

c)
```
SELECT Name, Abteilungsname
    FROM Person, Abteilung
    WHERE Abteilung=Abteilungsnr
```

5 SQL – Übungen

2) Geben Sie zu den folgenden, verbal formulierten Abfragen die passende SQL-Abfrage an. Die Abfragen müssen unabhängig vom konkreten Inhalt der Datenbank gültig sein!

   a) Liste aller Personen mit Name und Ort.

   b) Liste aller Personen mit einem Gehalt von mehr als 4.000,-- € mit Name und Gehalt, sortiert nach fallendem Gehalt (höchstes Gehalt zuerst).

5 SQL – Übungen

2) Geben Sie zu den folgenden, verbal formulierten Abfragen die passende SQL-Abfrage an. Die Abfragen müssen unabhängig vom konkreten Inhalt der Datenbank gültig sein!

c) Liste aller Orte mit der zusätzlichen Angabe, wie viele Personen von dort kommen.

d) Liste aller Abteilungen mit Abteilungsnamen und der Gehaltssumme (Summe über alle Gehälter der Personen dieser Abteilung). Der String "Gehaltssumme" soll als Überschrift der Spalte vorkommen.

5 SQL – Übungen

Die Tabellen unten sind für folgenden Use-Case:
Ein Fußballverband hat eine Datenbank mit den bei ihm gemeldeten Vereinen und Spielern. Alle Spieler haben eine vereinsübergreifend einmalige Pass-Nr. und können pro Saison (Attribut ist vom Typ String) nur bei einem Verein gemeldet sein.

Spieler

Pass_Nr	Name	Geburtsjahr
1	Peter Maier	1960
2	Klaus Müller	1980
3	Karl Hinz	1987
4	Thomas Kunze	1975

Verein

V_Nr	Name	Ort
1	Schlappen-Kicker 05	Heilbronn
2	Knochen-Knacker 09	Künzelsau
3	Ballermann 1955	Heilbronn
4	Alte Herren 1880	Schwäbisch Hall

Meldung

Spieler	Saison	Verein	Einsätze	Tore
1	00/01	1	6	3
2	00/01	3	12	1
3	00/01	2	1	0
1	99/00	2	6	4
4	00/01	1	2	1

5 SQL – Übungen

3) Geben Sie für folgende SQL-Abfragen an, welche Tabelle als Ergebnis erzeugt wird (Tabelle mit Überschriften):

i)
```
SELECT Verein.Name
    FROM Meldung, Verein, Spieler
    WHERE Verein=V_Nr AND
        Spieler=Pass_Nr AND
        Saison="00/01" AND
        Geburtsjahr>1962
    ORDER BY Verein.Name DESC
```

ii)
```
SELECT COUNT(*)
    FROM Meldung
    WHERE Saison="00/01"
```

5 SQL – Übungen

4) Geben Sie zu den folgenden, verbal formulierten Abfragen die passende SQL-Abfrage an. Die Abfragen müssen unabhängig vom konkreten Inhalt der Datenbank gültig sein.

   i) Liste aller Heilbronner Vereine mit Nr. und Namen, sortiert nach Nr.

   ii) Liste aller Spieler mit Pass-Nr. und Namen, die in der Saison 00/01 beim Verein "Knochen-Knacker 09" gespielt haben

   iii) Liste mit dem oder den (ev. auch mehrere) Torschützen-Königen der Saison 00/01 mit Namen und Anzahl der Toren

   iv) Liste der Vereine mit Nr. und Namen sowie der Anzahl dort in der Saison 00/01 gemeldeten Spieler